GitHub Copilot as a Coding-Agent Harness

Course Outline

Course Summary

GitHub Copilot is a coding-agent harness: the same agent loop, tools, context files, and approval model run inside VS Code, in a terminal through the GitHub Copilot CLI, and as a cloud agent on GitHub.com that takes an issue and returns a pull request. This session is a guided, hands-on tour of that harness for .NET and Java developers, showing what each surface does and how the three work together.

The session is organized around the three places Copilot runs. Participants begin in VS Code with the harness itself and project-aware chat, continue with agent mode shaped by repository instructions, custom agents, and MCP servers, move to GitHub.com to hand a task to the cloud agent and see Copilot code review, and finish in the terminal with the Copilot CLI. Each module pairs a 25-minute lesson and demonstration with a 25-minute hands-on exercise on sample .NET and Java projects, so participants work with the real product rather than with slides alone. The session closes with a comparison of Copilot, OpenCode, and Kiro as harnesses.

- **Duration:** Four hours, delivered live over video conference
- **Audience:** Developers and technical leads working in .NET or Java. No prior experience with GitHub Copilot chat, agent mode, the Copilot CLI, or the cloud agent is assumed.
- **Prerequisites:** Comfortable with Git, pull requests, and day-to-day work in VS Code.
- **Environment:** An enterprise GitHub account per participant with a Copilot seat that allows Copilot chat, agent mode, the Copilot CLI, and code review, and one virtual machine per participant with VS Code, the GitHub Copilot CLI, the GitHub CLI, Git, the .NET SDK, and a JDK. The instructor supplies the sample .NET and Java projects, published as public repositories that each participant copies into a private repository in their own account, and the pre-written issues used in the exercises. GitHub Actions and the cloud agent don't run in the personal repositories of enterprise accounts, so the instructor demonstrates the cloud agent, CI, and handing work from the terminal to the cloud.

Objectives

After this session, participants will be able to:

- Describe the Copilot harness in concrete terms: the model, the agent loop, the tools it can call, how context is assembled, how approvals work, and why the same harness appears in VS Code, the CLI, and GitHub.com.
- Use project-aware chat and agent mode in VS Code on a .NET or Java codebase, and shape the agent's behavior with repository instructions, custom agents, and MCP servers.
- Delegate a task to the Copilot cloud agent from an issue, follow its progress in the Agents panel, and use Copilot code review on the resulting pull request.
- Run agentic tasks from the terminal with the GitHub Copilot CLI, control approvals and sandboxing, and hand work between the terminal and the cloud agent.
- Summarize where GitHub Copilot stands out and where it is constrained as a coding-agent harness, compared with OpenCode and Kiro.

Topics

- Module 1 — The Copilot Harness and Chat in VS Code
- Module 2 — Agent Mode in VS Code with Instructions, Custom Agents, and MCP
- Module 3 — GitHub.com, the Cloud Agent and Code Review
- Module 4 — The GitHub Copilot CLI in the Terminal
- Conclusion — Copilot, OpenCode, and Kiro Side by Side

Module 1 — The Copilot Harness and Chat in VS Code

- A. Model versus harness: the agent loop, tools, context assembly, approvals, and session state, and why one harness shows up in VS Code, the Copilot CLI, the Copilot app, and GitHub.com
- B. The capability ladder: chat, agent mode, the Copilot CLI, and the cloud agent, and what changes at each step
- C. Chat that knows the project: adding files, selections, and the workspace as context, and the built-in actions for explaining, fixing, testing, and documenting code
- D. Model selection: the model picker, automatic model choice, and when a faster or a stronger model is the right call

- E. Hands-on activity: use chat with workspace context on the sample project to explain an unfamiliar area of the code, then generate a unit test for it and run it

Module 2 — Agent Mode in VS Code with Instructions, Custom Agents, and MCP

- A. Agent mode: how a task is planned, which tools the agent calls (file edits, terminal commands, tests, search), how approvals work, and how to read the session log
- B. Repository instructions: `.github/copilot-instructions.md`, path-scoped instruction files, and `AGENTS.md`, and when each is loaded
- C. Custom agents: defining a specialized agent under `.github/agents`, shared across VS Code, the CLI, and the cloud agent
- D. MCP servers: adding tools from the GitHub MCP server and from internal systems
- E. Hands-on activity: give agent mode a small, well-scoped change on the sample project (a new validation rule with unit tests), approve or reject each step, and review the diff; then add a repository instructions file and run the same task again to see how the instructions change the result

Module 3 — GitHub.com, the Cloud Agent and Code Review

- A. The Copilot cloud agent: assigning an issue, the Agents panel, how the agent runs in an isolated GitHub Actions environment, and the pull request it returns
- B. What happens inside a cloud run: self-review with Copilot code review, code scanning, secret scanning, dependency checks, and the session log
- C. Steering the cloud agent: writing issues it can act on, custom agents from the repository, and Copilot Spaces and Copilot Memory as context
- D. Copilot code review on pull requests: requesting a review, custom review instructions, and working through its comments
- E. Pull request summaries and Copilot Chat on GitHub.com for questions about a repository
- F. Running other coding agents from the Agents panel alongside the Copilot cloud agent
- G. Hands-on activity: write a small issue in your copy of the sample repository, have agent mode implement it on a branch, open a pull request, request a Copilot code review on it, and compare it with the cloud agent's pull request and session log from the demo

Module 4 — The GitHub Copilot CLI in the Terminal

- A. What the Copilot CLI is: the same coding-agent harness in a terminal, how it differs from the `gh` GitHub CLI, and the installation options (install script, Homebrew, WinGet, npm)
- B. Interactive and programmatic use: sessions, slash commands, model switching, and running prompts non-interactively for scripts and CI
- C. Permission and sandbox controls: previewing actions before they run, allow and deny rules, and the local sandbox
- D. Built-in GitHub integration: issues, pull requests, and the GitHub MCP server, plus custom MCP servers
- E. Reusing the same customization files: the instructions and custom agents from Module 2 work unchanged in the CLI
- F. Second opinions and handoffs: the built-in review agent, delegating a session to the cloud agent, and continuing a cloud session in the terminal
- G. Hands-on activity: from the terminal, point the Copilot CLI at a failing test in the sample project that is described in a GitHub issue; have it diagnose and fix the problem under step-by-step approval, run the test suite, and open a pull request without leaving the terminal

Conclusion — Copilot, OpenCode, and Kiro Side by Side

- A. How each harness handles model choice and where the model runs, context through instruction files and MCP servers, approvals and sandboxing, the path from a task to a pull request, and which parts of each are open source
- B. Where GitHub Copilot stands out: one harness across the IDE, the terminal, and GitHub.com; an issue-to-pull-request flow with code review and security scanning built in; a model picker spanning several vendors; and instructions, custom agents, and MCP configuration shared across all three surfaces
- C. Where GitHub Copilot is constrained: the cloud agent works only with repositories hosted on GitHub, one repository per task with a time cap per session on GitHub Actions; the model menu is set by GitHub; and the agent services are proprietary, with only client-side pieces such as VS Code (which now includes Copilot Chat) and the Copilot SDK open source